

```
<!--Programación Lógica y Funcional-->
```

OcaIm

{

```
<Por="Marcos Daniel Garcia Rodriguez"
```

```
"Gomez Cuevas Carlos"
```

```
"Grande Ortega Maximiliano"
```

```
"Gonzalez Cristobal Omar"/>
```



Contenidos

- 01 Contexto e Historia
- 02 Modelo de computo
- 03 Sintaxis
- 04 Sistemas de tipos / runtime
- 05 Demo en vivo
- 06 Caso real

Historia y Creadores {

* Año y origen: Surgió en 1996 en el instituto de investigación INRIA (Francia). Evolucionó de la familia de lenguajes ML y Caml (creado en 1987).

* Creadores: Xavier Leroy, Jérôme Vouillon, Damien Doligez, Didier Rémy y Ascánder Suárez. (Xavier Leroy suele ser reconocido como el arquitecto principal).



}

¿Por qué Meta escribe
sus analizadores en
OCaml? {

Meta usa OCaml como base para sus herramientas de análisis estático más críticas: Flow (para JavaScript), Hack/HHVM (para PHP) e Infer (para Java, C, C++). Las razones técnicas principales son:

Seguridad y rigor matemático: Gracias a su fuerte sistema de tipos estático, los ingenieros de Meta pueden crear herramientas de verificación formal muy complejas (basadas en lógica de separación) sabiendo que el propio compilador de OCaml prevendrá los errores.

Ideal para analizar código: OCaml fue diseñado con características matemáticas que lo hacen excepcional para escribir compiladores y analizadores de código. Herramientas nativas del lenguaje como el pattern matching (coincidencia de patrones) y los tipos de datos algebraicos hacen que desarmar y evaluar el código de otros lenguajes sea un proceso muy natural.



}

Modelo de Computo

El cómputo en OCaml se sustenta formalmente en el cálculo lambda tipado, enriquecido con semántica imperativa y un sistema de módulos de orden superior.

Estrategia de evaluación por valor (Call-by-value / Eagle)

Evaluación estricta.

Todos los argumentos de una expresión o llamada a función se evalúan por completo antes de invocar la abstracción.

Orden de evaluación de argumentos.

El estándar del lenguaje deja el orden sin especificar a nivel conceptual, pero la implementación oficial del compilador (bytecode y nativo) los evalúa de derecha a izquierda.

Evaluación perezosa explícita.

A diferencia de Haskell (donde toda la evaluación es perezosa por defecto), OCaml solo posterga el cómputo cuando se solicita expresamente mediante la primitiva `lazy expr` y se fuerza con `Lazy.force` del módulo de la biblioteca estándar.

}

Sintaxis

Vinculacion de nombres y operadores.

Let.

Las variables son inmutables por defecto, una vez definido, se queda así para siempre.

```
let edad = 20
```

Definicion de funciones y Currificacion.

rec.

Las funciones admiten currificacion implicita. Cuando una funcion requiere recursion, se especifica con **rec**.

```
let multiplicar a b = a * b  
let duplicar = multiplicar 2
```

```
let rec factorial n =  
  if n <= 1 then 1  
  else n * factorial (n - 1)
```

}

Tipos algebraicos de datos

Es un tipo personalizado que define una lista cerrada de formas posibles que puede tomar un dato

Booleanos.

ADT con solo dos constructores.

```
type bool = true | false
```

Option.

Se usa para manejar valores ausentes o nulos de forma segura.

```
type 'a option = None | Some of 'a
```

Resultado.

Diseñado para el manejo funcional de errores sin lanzar excepciones.

```
type ('a, 'b) result = Ok of 'a | Error of 'b
```

Listas.

Internamente estan definidas como un tipo recursivo algebraico.

```
type 'a list = [] | (::) of 'a * 'a list
```

}

Sintaxis

Pattern Matching

Match.

Es el mecanismo de coincidencia de patrones (pattern matching). **match** evalúa el valor, examina su forma o estructura, extrae sus partes internas y decide qué camino tomar en una sola operación.

Permite definir variantes (uniones discriminadas) y registros con verificación exhaustiva de patrones.

```
let saludar idioma =  
  match idioma with  
  | "es" → "Hola"  
  | "en" → "Hello"  
  | "fr" → "Bonjour"  
  | _    → "No conozco ese idioma"
```

Sintaxis imperativa

Como en OCaml casi todo es inmutable. para programar de forma imperativa se necesita de:

- **Variables que puedan cambiar**(**ref**, **!**, **:=**).
- **Punto y coma.**
- **Ciclos tradicionales**(**for** y **while**).

```
let total = ref 0  
  
let () =  
  for i = 1 to 5 do  
    total := !total + i  
  done;  
  
print_string "La suma es: ";  
print_int !total
```

}

Inferencia sin anotaciones

```
let suma a b = a + b  
(* val suma : int -> int -> int *)
```

Se pueden anotar tipos si se desea, aunque es opcional, ya que el compilador los deduce (Hindley-Milner)

option frente a null

```
type 'a option = None | Some of 'a
```

None = no hay nada.
Some x = hay un valor, y es x.

}

Sistema de módulos

Firma

```
module type EDAD = sig
  type t
  val crear : int -> t option
end
```

Implementación

```
module Edad : EDAD = struct
  type t = int
  let crear n =
    if n >= 0 && n <= 150
    then Some n else None
  end
end
```

Módulo String
Sabe comparar

Functor Set.Make
Función de módulos

ConjuntoStrings
Conjunto de strings

}

DEMO URL {

DEMO

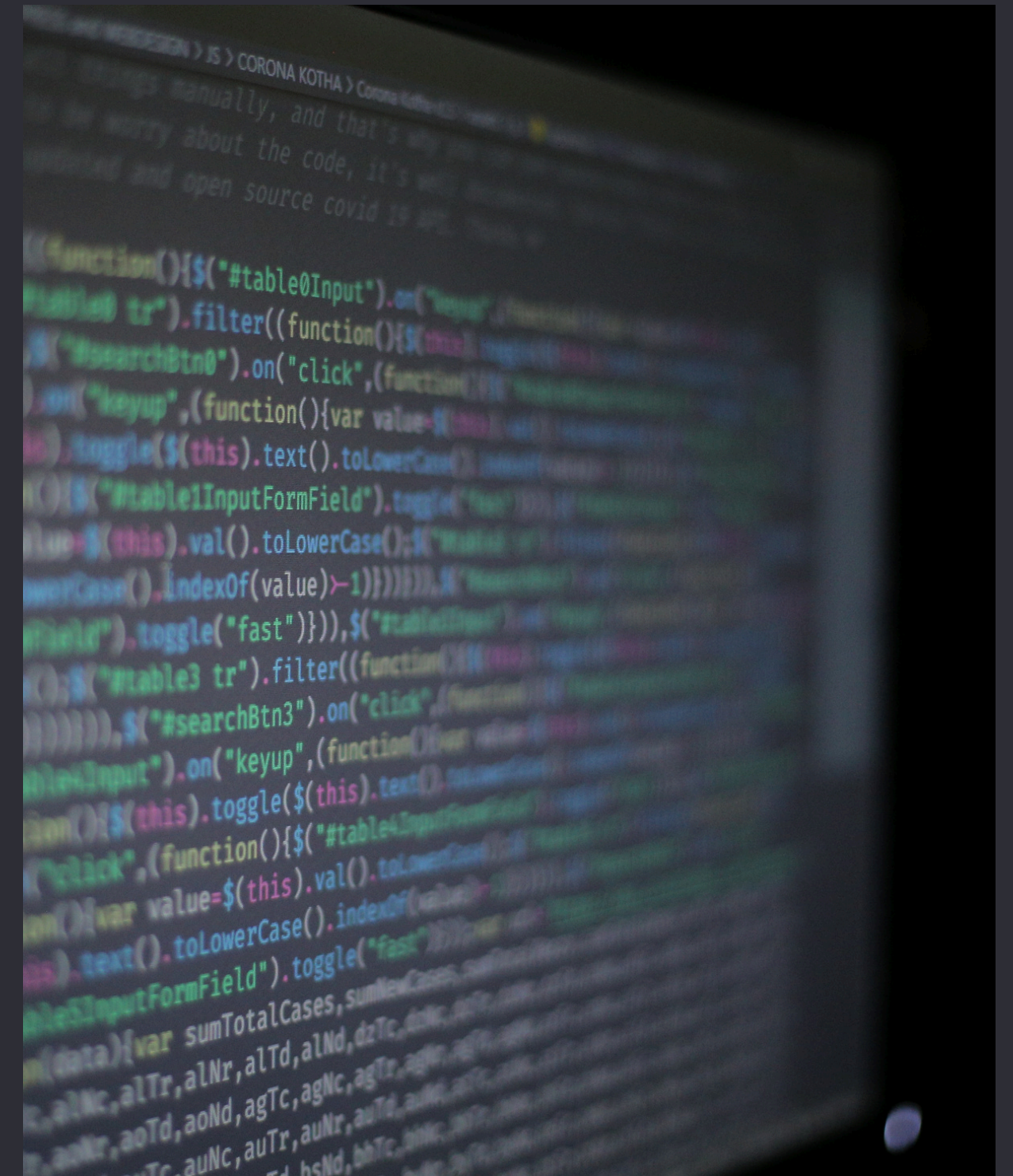
<https://asciinema.org/a/5zP3Mu0M4FmeVFiz>

}

CASO REAL {

VPNKit El subproyecto de código abierto más importante dentro de Docker que usa OCaml se llama VPNKit

. ¿Qué hace? Actúa como un traductor que intercepta el tráfico de red de los contenedores a nivel de usuario. Cuando un contenedor intenta abrir una conexión TCP/IP, VPNKit toma esos paquetes de red crudos en OCaml y los convierte en llamadas de red estándar del sistema operativo nativo (macOS o Windows).

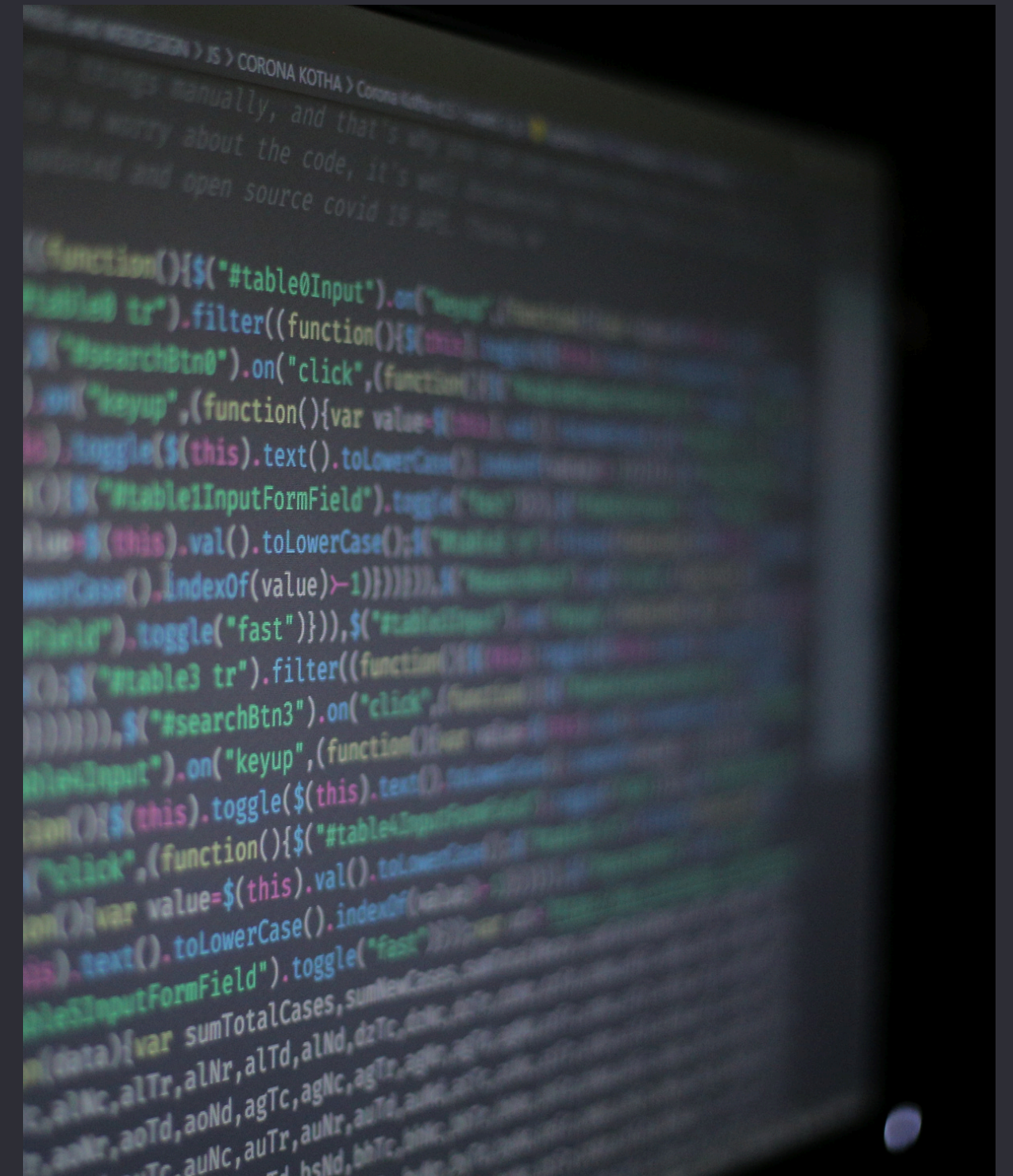


}

CASO REAL {

¿Por qué OCaml?

Reescribir toda una pila de protocolos de red (TCP, UDP, DNS, HTTP) en un lenguaje de alto nivel requería una seguridad absoluta en el manejo de memoria. OCaml ofrece la velocidad del código nativo compilado combinada con un sistema de tipos matemáticamente riguroso que evita desbordamientos de búfer (buffer overflows) y fallos de seguridad críticos que ocurren comúnmente en C o C++.



}